AI-ASSISTED WEB APPLICATION ESE

CampusFlow
Student Management Portal

Project Report and Prompt Book

Prepared by	[Enter your full name]
Division / GR No.	[Enter division and GR number]
Subject	AI-Assisted Web Application
Submission	End Semester Examination
Date	29 October 2026

Before submitting: replace the two bracketed fields above and rename this PDF in your college's required format, for example 3EN1_123456_Rahul.pdf.

1. Project overview

CampusFlow is a responsive Student Management Portal prototype for a college administrator. It presents student records in one place, gives a quick attendance view, and supports simple daily actions. The project focuses on prompt engineering and AI-assisted planning, design, improvement, testing, and explanation rather than advanced programming.

Problem statement

In many small college workflows, student lists and attendance may be kept in separate spreadsheets or paper registers. Finding a student record, checking daily attendance, or preparing a simple list can take unnecessary time. The prototype brings these high-frequency tasks into one clear interface.

Objective

To create a simple, polished, beginner-friendly web prototype that allows an administrator to review student information, add a student, change attendance status, and download a student directory report.

Target users

- College administrator - maintains basic records and checks attendance.
- Faculty member - can view students and update their daily status in a future version.
- Student - an indirect user who benefits from accurate records in a future version.

Scope of this prototype

- Dashboard with summary cards, an attendance chart, recent activity, and quick actions.
- Students page with search, year filter, and an Add Student form.
- Attendance page where each status cycles through Present, Late, and Absent.
- Reports page that downloads the student directory as a CSV file.

The prototype uses sample data only. It does not use personal student information.

2. Requirements and design

Functional requirements

Requirement	How it is met in the prototype
View portal overview	Dashboard displays total students, attendance, pending requests, events, chart, and activity.
Find a student	Search checks name, student ID, or programme; a second control filters by year.
Create a record	Add Student opens a form with required name, email, programme, and year fields.
Record attendance	The Attendance page changes a student's state between Present, Late, and Absent.
Obtain a report	Download CSV creates a spreadsheet-ready student directory in the browser.

Non-functional requirements

- Responsive: cards and controls adapt for tablets and phones.
- Easy to use: clear labels, readable contrast, simple navigation, and immediate confirmation messages.
- Portable: runs from three standard files (HTML, CSS, JavaScript) without installing a server or database.
- Safe for demonstration: uses sample records and saves changes only in the current browser.

Interface plan

The visual direction is a navy left sidebar, white content cards, blue and teal action colors, and rounded controls. The dashboard is designed to give information first, while the Students and Attendance pages make actions easy to discover. On a smaller screen, the sidebar becomes a menu and multiple columns stack vertically.

3. AI-assisted development approach

The application was planned with prompts that described the user, task, constraints, desired visual style, and success criteria. Broad first prompts were refined into specific prompts. This is important because AI output becomes more useful when the instructions state what to include and what to avoid.

Technology used

Technology	Purpose
HTML	Creates the page structure: navigation, dashboard, tables, modal form, and report controls.
CSS	Creates the responsive visual design, including desktop and mobile breakpoints.
JavaScript	Controls navigation, search/filtering, form submission, attendance changes, browser storage, and CSV export.
AI assistant	Helped plan the problem, interface, interactions, tests, prompt improvements, and report.

Core implementation choices

- Sample student data makes the dashboard meaningful immediately.
- Browser local storage remembers records and attendance changes on the same device.
- Form validation uses required inputs and email validation before a new record is saved.
- A CSV Blob is generated in the browser, so the user can download a simple report without a backend.
- No external library or paid tool is required, which makes the project easy for a beginner to explain.

Important AI improvement

The initial requirement could have produced a static design. The refined prompt explicitly asked for working search, a validated form, status changes, local storage, CSV export, and responsive behavior. This changed the result from a visual mock-up into a usable prototype.

4. Prompt book

The complete copyable prompt book is included as a separate Markdown file. The key prompt engineering evidence is summarized below.

Stage	Initial prompt	Improved prompt and reason
Plan	Suggest a simple ESE web app topic for a beginner.	Recommend a Student Management Portal with dashboard, student records, attendance, and report. The scope becomes clear and manageable.
Design	Plan a modern portal interface.	Use a navy sidebar, white cards, blue/teal accents, accessible labels, and a mobile menu. The visual direction becomes consistent.
Build	Generate a front-end student portal.	Use only HTML, CSS, and JavaScript; make search, form, attendance, local storage, and CSV export work without a server. The result becomes interactive.
Test	Review the portal for issues.	Check navigation, form validation, filtering, attendance, persistence, export, and phone layout; suggest only important fixes. The review becomes focused.
Report	Write a project report.	Include problem, objective, users, requirements, prompt process, testing, limitations, conclusion, and publishing steps. The report matches the ESE task.

Prompt-engineering reflection

The most useful technique was adding constraints such as beginner-friendly, responsive, working interactions, no database, browser storage, and no external libraries. These constraints improved both the quality of the prototype and the student's ability to explain it.

5. Testing and improvements

Testing was planned around the features that a lecturer can see during a short demonstration. A code-level verification checked that the project files, responsive CSS rules, student form, search, attendance update, export action, and local storage wiring are present. The following table is also a manual demonstration checklist.

Test	Action	Expected result	Result
Open portal	Open index.html in a browser.	Dashboard loads with sample data.	Pass - page structure and scripts verified.
Navigation	Select Students, Attendance, and Reports.	Selected page displays and menu highlight changes.	Pass - navigation handlers verified.
Search / filter	Type a name or select a year.	Only matching records remain in the table.	Pass - filter function verified.
Add student	Complete and submit the form.	Required fields are checked; record is added and saved.	Pass - form and storage wiring verified.
Attendance	Click Change status for a student.	Status cycles and summary totals update.	Pass - status update function verified.
Export	Click Download CSV.	A CSV file is downloaded with student records.	Pass - CSV generator verified.
Phone layout	Open on a narrow screen.	Sidebar becomes a menu; cards stack in one column.	Pass - mobile CSS breakpoints verified.

Fixes and refinements applied

- Added email and required-field validation to avoid incomplete student records.
- Made the search more useful by checking multiple fields, not only student name.
- Updated dashboard attendance figures when attendance status changes.
- Added persistent browser storage and clear toast messages after actions.
- Added smaller-screen rules so the portal remains usable on a phone.

6. Limitations, conclusion, and publishing

Current limitations

- The application is a front-end prototype. It has no secure sign-in or role-based access.
- Data is stored in one browser, not in a shared online database. Clearing browser data can remove changes.
- The student list is sample data; a real system would connect to a secure college database.
- There is no edit/delete screen, automated attendance import, or analytics for a full production system.

Future enhancement

A future version could add login for administrators and faculty, a secure database, edit and delete actions, student login, email notifications, and attendance reports by subject or month.

Conclusion

CampusFlow demonstrates how AI can help a beginner plan, design, build, improve, test, and explain a web application prototype. The final product solves a clear college administration problem through a polished dashboard, student records, attendance updates, and a downloadable report. The most important lesson from this project is that specific prompts and practical constraints lead to better AI-assisted results.

How to publish a live link with GitHub Pages

Step	What to do
1	Create a free public GitHub repository named campusflow-student-portal.
2	Upload index.html, styles.css, and app.js from the project folder and commit the files.
3	Open Settings, then Pages. Select Deploy from a branch, choose main and /(root), then save.
4	Wait one or two minutes, then copy the GitHub Pages URL. Open it on a computer and phone before pasting it into the Google Form.

Submission checklist

- Replace student name, division, and GR number on the cover page.
- Rename the PDF to the format requested by the college, for example 3EN1_123456_Rahul.pdf.
- Open index.html and do the short demonstration once before submitting.
- Publish with GitHub Pages if the Google Form requires a live link, then paste that link in the form.